In [32]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.gridspec import GridSpec
import warnings
```

In [2]:
```
np.random.seed(42)

num_users_per_group = 5000

start_date = datetime(2023, 11, 1)

test_duration_days = 14 # 2 week test



# Control Group (A)

control_user_ids = [f'UserA_{10000+i}' for i in range(num_users_per_group)]

control_group_assignment = ['Control'] * num_users_per_group

# Assume a baseline conversion rate of 10% for control

control_conversions = np.random.binomial(1, 0.10, num_users_per_group)

control_clicks = np.random.randint(0, 15, num_users_per_group) # Clicks per user

control_page_views = control_clicks + np.random.randint(1, 5, num_users_per_group)

control_page_views = np.maximum(1, control_page_views) # At least 1 page view



# Treatment Group (B) - let's assume a slight lift

treatment_user_ids = [f'UserB_{20000+i}' for i in range(num_users_per_group)]

treatment_group_assignment = ['Treatment'] * num_users_per_group

# Assume a target conversion rate of 11.5% for treatment (1.5% lift)

treatment_conversions = np.random.binomial(1, 0.115, num_users_per_group)

treatment_clicks = np.random.randint(0, 17, num_users_per_group) # Slightly more cl

treatment_page_views = treatment_clicks + np.random.randint(1, 5, num_users_per_gro

treatment_page_views = np.maximum(1, treatment_page_views)
```

```
# Combine data
all_user_ids = control_user_ids + treatment_user_ids
all_groups = control_group_assignment + treatment_group_assignment
all_conversions = np.concatenate([control_conversions, treatment_conversions])
all_clicks = np.concatenate([control_clicks, treatment_clicks])
all_page_views = np.concatenate([control_page_views, treatment_page_views])

# Assign random dates within the test period
all_dates = [ (start_date + timedelta(days=np.random.randint(0, test_duration_days)
              for _ in range(num_users_per_group * 2) ]

df_ab_test = pd.DataFrame({
    'UserID': all_user_ids,
    'Group': all_groups,
    'Date': all_dates,
    'PageViews': all_page_views,
    'Clicks': all_clicks,
    'Converted': all_conversions
})

# Ensure clicks are not more than page views (though unlikely with current generati
df_ab_test['Clicks'] = df_ab_test.apply(lambda row: min(row['Clicks'], row['PageVie
```

In [3]:

```
# Save to CSV
df_ab_test.to_csv('ab_test_results_mock_data.csv', index=False)
print("Mock A/B test data generated: ab_test_results_mock_data.csv")
```

```
print(df_ab_test.sample(10, random_state=42))
```

```
Mock A/B test data generated: ab_test_results_mock_data.csv
           UserID      Group        Date  PageViews  Clicks  Converted
6252  UserB_21252  Treatment  2023-11-07          8       4          0
4684  UserA_14684    Control  2023-11-10          5       1          0
1731  UserA_11731    Control  2023-11-04          8       6          0
4742  UserA_14742    Control  2023-11-12          3       2          0
4521  UserA_14521    Control  2023-11-12         12       8          0
6340  UserB_21340  Treatment  2023-11-03          7       3          0
576   UserA_10576    Control  2023-11-08          4       0          0
5202  UserB_20202  Treatment  2023-11-05         15      14          1
6363  UserB_21363  Treatment  2023-11-12         16      14          1
439   UserA_10439    Control  2023-11-08          9       8          0
```

In [5]:
```
# Basic check of conversion rates

print("\nOverall Conversion Rates by Group:")
print(df_ab_test.groupby('Group')['Converted'].mean())
```

```
Overall Conversion Rates by Group:
Group
Control      0.0958
Treatment    0.1180
Name: Converted, dtype: float64
```

Hypothesis:

Null (H_0): The new feature has no effect on conversion rate (Control = Treatment)

Alternative (H_1): The new feature increases conversion rate (Treatment > Control)

Key details from the data generation code:

Control baseline conversion rate: 10%

Treatment target conversion rate: 11.5% (a 1.5% absolute lift)

Users per group: 5,000 (10,000 total)

Test duration: 14 days (Nov 1–14, 2023)

Key metric: Conversion Rate (primary), Clicks & PageViews (secondary)

In [6]:
```
# Data Loading & Cleaning
# Basic shape and structure
print("Shape:", df_ab_test.shape)
print("\nColumn types:\n", df_ab_test.dtypes)
print("\nFirst 5 rows:\n", df_ab_test.head())
```

```
Shape: (10000, 6)

Column types:
 UserID       object
Group        object
Date         object
PageViews     int32
Clicks        int64
Converted     int32
dtype: object

First 5 rows:
         UserID    Group        Date  PageViews  Clicks  Converted
0  UserA_10000  Control  2023-11-08          5       2          0
1  UserA_10001  Control  2023-11-14          8       7          1
2  UserA_10002  Control  2023-11-03          7       6          0
3  UserA_10003  Control  2023-11-06          6       2          0
4  UserA_10004  Control  2023-11-03          6       5          0
```

In [7]:
```
print("\nMissing values:\n", df_ab_test.isnull().sum())
```

```
Missing values:
 UserID       0
Group        0
Date         0
PageViews    0
Clicks       0
Converted    0
dtype: int64
```

In [8]:
```
duplicate_users = df_ab_test[df_ab_test.duplicated(subset='UserID', keep=False)]
print(f"\nDuplicate UserIDs: {len(duplicate_users)}")
```

```
Duplicate UserIDs: 0
```

In [9]:
```
users_in_both = (
    df_ab_test.groupby('UserID')['Group']
    .nunique()
    .reset_index()
    .query('Group > 1')
)
print(f"Users in both groups: {len(users_in_both)}")
```

```
Users in both groups: 0
```

In [11]:
```
print("\nGroup sizes:")
print(df_ab_test['Group'].value_counts())
```

```
Group sizes:
Group
Control      5000
Treatment    5000
Name: count, dtype: int64
```

In [13]:
```
#Sanity check — Clicks should never exceed PageViews
invalid_rows = df_ab_test[df_ab_test['Clicks'] > df_ab_test['PageViews']]
print(f"\nRows where Clicks > PageViews: {len(invalid_rows)}")
```

```
Rows where Clicks > PageViews: 0
```

In [14]:
```
df_ab_test['Date'] = pd.to_datetime(df_ab_test['Date'])
print(f"\nDate range: {df_ab_test['Date'].min()} to {df_ab_test['Date'].max()}")
```

```
Date range: 2023-11-01 00:00:00 to 2023-11-14 00:00:00
```

In [15]:
```
#Key Metrics
#Group-level aggregation
metrics = df_ab_test.groupby('Group').agg(
    total_users    = ('UserID', 'count'),
    total_conversions = ('Converted', 'sum'),
    total_clicks   = ('Clicks', 'sum'),
    total_pageviews = ('PageViews', 'sum')
).reset_index()

#Derived metrics
metrics['conversion_rate']  = metrics['total_conversions'] / metrics['total_users']
metrics['ctr']              = metrics['total_clicks'] / metrics['total_pageviews']

print(metrics.to_string(index=False))
```

```
    Group  total_users  total_conversions  total_clicks  total_pageviews  conversion
_rate      ctr
  Control         5000                479         34674            47263
0.0958 0.733639
Treatment         5000                590         40331            52781
0.1180 0.764120
```

In [16]:
```
#Observed difference
control   = metrics[metrics['Group'] == 'Control'].iloc[0]
treatment = metrics[metrics['Group'] == 'Treatment'].iloc[0]

abs_diff    = treatment['conversion_rate'] - control['conversion_rate']
rel_lift    = abs_diff / control['conversion_rate'] * 100

print(f"\nAbsolute difference : {abs_diff:.4f}  ({abs_diff*100:.2f}%)")
print(f"Relative lift       : {rel_lift:.2f}%")
```

```
Absolute difference : 0.0222  (2.22%)
Relative lift       : 23.17%
```

In [18]:
```
#Z-Test for proportions
n_control   = int(control['total_users'])
n_treatment = int(treatment['total_users'])
conv_control   = int(control['total_conversions'])
conv_treatment = int(treatment['total_conversions'])

# 4a. Z-test
count  = np.array([conv_treatment, conv_control])
nobs   = np.array([n_treatment, n_control])

z_stat, p_value = proportions_ztest(count, nobs, alternative='larger')  # one-taile

print(f"Z-statistic : {z_stat:.4f}")
print(f"P-value     : {p_value:.4f}")
```

```
Z-statistic : 3.5924
P-value     : 0.0002
```

In [19]:
```
#Confidence interval for each group's conversion rate
alpha = 0.05

ci_control   = proportion_confint(conv_control,   n_control,   alpha=alpha, method=
ci_treatment = proportion_confint(conv_treatment, n_treatment, alpha=alpha, method=

print(f"\nControl   95% CI : ({ci_control[0]:.4f}, {ci_control[1]:.4f})")
print(f"Treatment 95% CI : ({ci_treatment[0]:.4f}, {ci_treatment[1]:.4f})")
```

```
Control   95% CI : (0.0876, 0.1040)
Treatment 95% CI : (0.1091, 0.1269)
```

In [27]:
```
#Decision
print("\n Decision")
if p_value < alpha:
    print(f"p={p_value:.4f} < alpha={alpha} → REJECT H₀")
    print("The treatment has a statistically significant effect on conversion rate.
else:
    print(f"p={p_value:.4f} >= alpha={alpha} → FAIL TO REJECT H₀")
    print("No statistically significant difference detected.")
```

```
 Decision
p=0.0002 < alpha=0.05 → REJECT H₀
The treatment has a statistically significant effect on conversion rate.
```

In [22]:
```
#Practical Significance & Power Analysis
#Effect size (Cohen's h for proportions)
effect_size = proportion_effectsize(treatment['conversion_rate'], control['conversi
print(f"Effect size (Cohen's h) : {effect_size:.4f}")
print("(Small ≈ 0.2, Medium ≈ 0.5, Large ≈ 0.8)")
```

```
Effect size (Cohen's h) : 0.0719
(Small ≈ 0.2, Medium ≈ 0.5, Large ≈ 0.8)
```

In [24]:
```
# Practical significance(is the lift worth acting on?)
# Rule of thumb: define a Minimum Detectable Effect (MDE) your business cares about
MDE = 0.01  # 1 percentage point lift = meaningful for this business

print(f"\nObserved absolute lift : {abs_diff*100:.2f}%")
print(f"Minimum meaningful lift: {MDE*100:.2f}%")

if abs_diff >= MDE:
    print("Lift exceeds MDE → Practically significant")
else:
    print("Lift below MDE → Not practically significant")
```

```
Observed absolute lift : 2.22%
Minimum meaningful lift: 1.00%
Lift exceeds MDE → Practically significant
```

In [25]:
```
# Power analysis(did we have enough users to detect this effect?)
analysis = NormalIndPower()
power = analysis.solve_power(
    effect_size = effect_size,
    nobs1       = n_control,
```

```
    alpha       = alpha,
    alternative = 'larger'
)

print(f"\nAchieved statistical power : {power:.4f}")
print("(Target is typically ≥ 0.80)")
```

```
Achieved statistical power : 0.9745
(Target is typically ≥ 0.80)
```

In [26]:

```
# Required sample size to detect the MDE at 80% power
required_n = analysis.solve_power(
    effect_size = proportion_effectsize(
                    control['conversion_rate'] + MDE,
                    control['conversion_rate']),
    alpha       = alpha,
    power       = 0.80,
    alternative = 'larger'
)

print(f"\nRequired users per group to detect {MDE*100}% lift at 80% power: {int(np.
print(f"Actual users per group: {n_control}")
```

```
Required users per group to detect 1.0% lift at 80% power: 11202
Actual users per group: 5000
```

In [28]:

```
#Interpretation & Segmentation

#clean summary report
print("=" * 55)
print("         A/B TEST RESULTS SUMMARY REPORT")
print("=" * 55)

print(f"""
EXPERIMENT OVERVIEW
  Duration       : {df_ab_test['Date'].min().date()} to {df_ab_test['Date'].max().d
  Total users    : {len(df_ab_test):,}
  Users/group    : {n_control:,}

CONVERSION RATES
  Control        : {control['conversion_rate']:.2%}
  Treatment      : {treatment['conversion_rate']:.2%}
  Absolute lift  : +{abs_diff:.2%}
  Relative lift  : +{rel_lift:.1f}%

STATISTICAL TEST (One-tailed Z-test, α=0.05)
  Z-statistic    : {z_stat:.4f}
  P-value        : {p_value:.4f}
  Significant    : {'YES ✓' if p_value < alpha else 'NO ✗'}

CONFIDENCE INTERVALS (95%)
  Control        : ({ci_control[0]:.2%}, {ci_control[1]:.2%})
  Treatment      : ({ci_treatment[0]:.2%}, {ci_treatment[1]:.2%})

PRACTICAL SIGNIFICANCE
  Effect size    : {effect_size:.4f} (Small)
  Power achieved : {power:.2%}
```

```
  MDE threshold  : 1.00%
  Exceeds MDE    : {'YES ✓' if abs_diff >= MDE else 'NO ✗'}

VERDICT: SHIP THE FEATURE
""")
print("=" * 55)
```

```
=======================================================
         A/B TEST RESULTS SUMMARY REPORT
=======================================================

EXPERIMENT OVERVIEW
  Duration       : 2023-11-01 to 2023-11-14
  Total users    : 10,000
  Users/group    : 5,000

CONVERSION RATES
  Control        : 9.58%
  Treatment      : 11.80%
  Absolute lift  : +2.22%
  Relative lift  : +23.2%

STATISTICAL TEST (One-tailed Z-test, α=0.05)
  Z-statistic    : 3.5924
  P-value        : 0.0002
  Significant    : YES ✓

CONFIDENCE INTERVALS (95%)
  Control        : (8.76%, 10.40%)
  Treatment      : (10.91%, 12.69%)

PRACTICAL SIGNIFICANCE
  Effect size    : 0.0719 (Small)
  Power achieved : 97.45%
  MDE threshold  : 1.00%
  Exceeds MDE    : YES ✓

VERDICT: SHIP THE FEATURE

=======================================================
```

In [29]:

```
#Segmentation by date(did the effect grow or stay stable over time?)
daily = df_ab_test.groupby(['Date', 'Group'])['Converted'].agg(['sum', 'count']).re
daily.columns = ['Date', 'Group', 'conversions', 'users']
daily['conversion_rate'] = daily['conversions'] / daily['users']

print("\nDaily Conversion Rates by Group:")
print(daily.pivot(index='Date', columns='Group', values='conversion_rate').round(4)
```

```
Daily Conversion Rates by Group:
Group       Control  Treatment
Date
2023-11-01   0.0951     0.1250
2023-11-02   0.1182     0.1152
2023-11-03   0.0753     0.1048
2023-11-04   0.0806     0.1225
2023-11-05   0.0982     0.0929
2023-11-06   0.0939     0.1058
2023-11-07   0.0947     0.1425
2023-11-08   0.1108     0.1176
2023-11-09   0.0659     0.1200
2023-11-10   0.1003     0.1304
2023-11-11   0.0981     0.1090
2023-11-12   0.0984     0.1395
2023-11-13   0.1079     0.1138
2023-11-14   0.1054     0.1152
```

In [30]:

```
#Segmentation by engagement level
# split users into Low / Medium / High based on PageViews
df_ab_test['engagement'] = pd.cut(
    df_ab_test['PageViews'],
    bins=[0, 4, 9, 100],
    labels=['Low (1-4)', 'Medium (5-9)', 'High (10+)']
)

engagement_seg = df_ab_test.groupby(['engagement', 'Group']).agg(
    users       = ('UserID', 'count'),
    conversions = ('Converted', 'sum')
).reset_index()

engagement_seg['conversion_rate'] = engagement_seg['conversions'] / engagement_seg[

print("\nConversion Rate by Engagement Segment:")
print(engagement_seg.pivot(
    index='engagement', columns='Group', values='conversion_rate'
).round(4).to_string())
```

```
Conversion Rate by Engagement Segment:
Group         Control  Treatment
engagement
Low (1-4)      0.1092     0.1234
Medium (5-9)   0.0951     0.1161
High (10+)     0.0916     0.1176
```

```
C:\Users\Hp\AppData\Local\Temp\ipykernel_13892\1137797557.py:9: FutureWarning: The d
efault of observed=False is deprecated and will be changed to True in a future versi
on of pandas. Pass observed=False to retain current behavior or observed=True to ado
pt the future default and silence this warning.
  engagement_seg = df_ab_test.groupby(['engagement', 'Group']).agg(
```

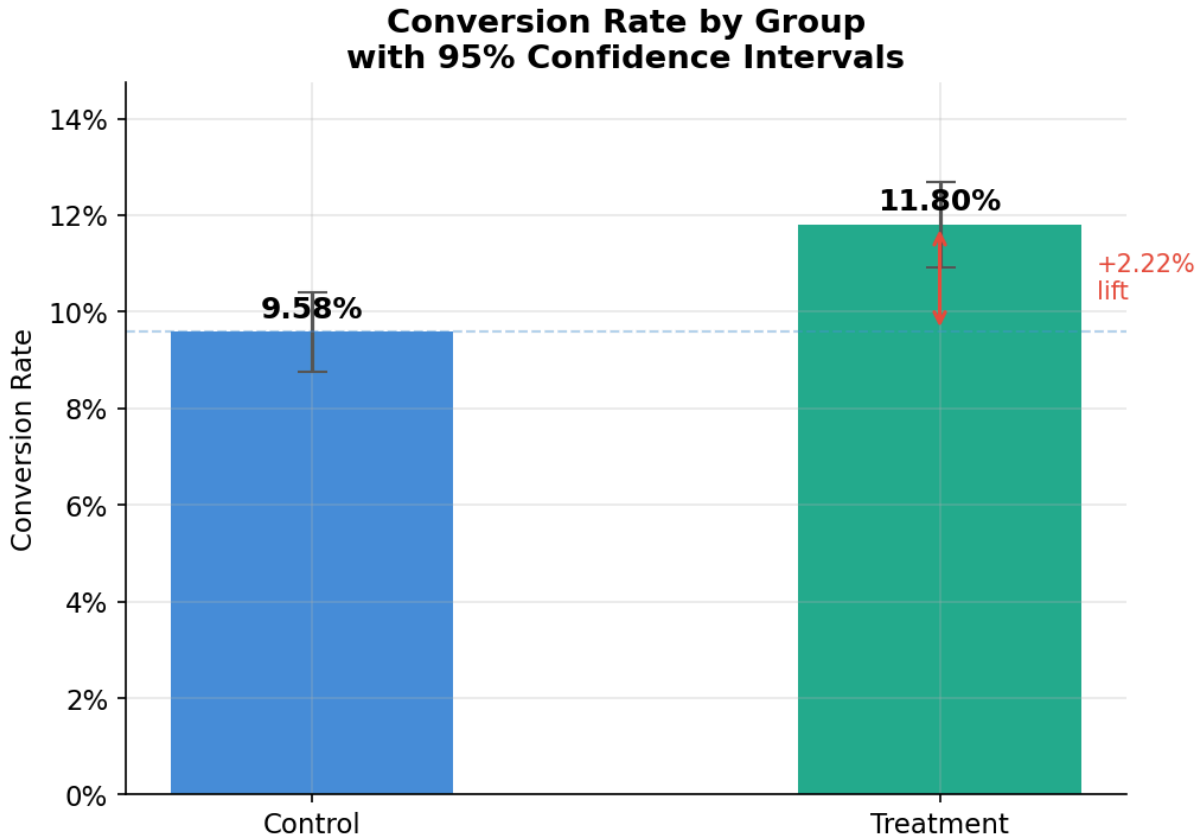
In [33]:

```
#Visualizations
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.size': 11,
```

```
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'figure.dpi': 150
})

BLUE  = '#4A90D9'   # Control
TEAL  = '#27AE8F'   # Treatment
GRAY  = '#BDC3C7'
RED   = '#E74C3C'
```

In [34]:

```
fig, ax = plt.subplots(figsize=(7, 5))

groups = ['Control', 'Treatment']
rates  = [control['conversion_rate'], treatment['conversion_rate']]
errors = [
    (ci_control[1]  - ci_control[0])  / 2,
    (ci_treatment[1] - ci_treatment[0]) / 2
]
colors = [BLUE, TEAL]

bars = ax.bar(groups, rates, color=colors, width=0.45,
              yerr=errors, capsize=6, error_kw=dict(elinewidth=1.5, ecolor='#555'))

for bar, rate in zip(bars, rates):
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 0.003,
            f'{rate:.2%}', ha='center', fontweight='bold', fontsize=12)

ax.axhline(y=control['conversion_rate'], color=BLUE, linestyle='--', alpha=0.4, lin
ax.set_ylabel('Conversion Rate')
ax.set_title('Conversion Rate by Group\nwith 95% Confidence Intervals', fontweight=
ax.set_ylim(0, max(rates) * 1.25)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))

# Annotate lift
ax.annotate('', xy=(1, rates[1]), xytext=(1, rates[0]),
            arrowprops=dict(arrowstyle='<->', color=RED, lw=1.5))
ax.text(1.25, (rates[0] + rates[1]) / 2,
        f'+{abs_diff:.2%}\nlift', color=RED, fontsize=10, va='center')

plt.tight_layout()
plt.savefig('plot1_conversion_rates.png', bbox_inches='tight')
plt.show()
print("Saved: plot1_conversion_rates.png")
```

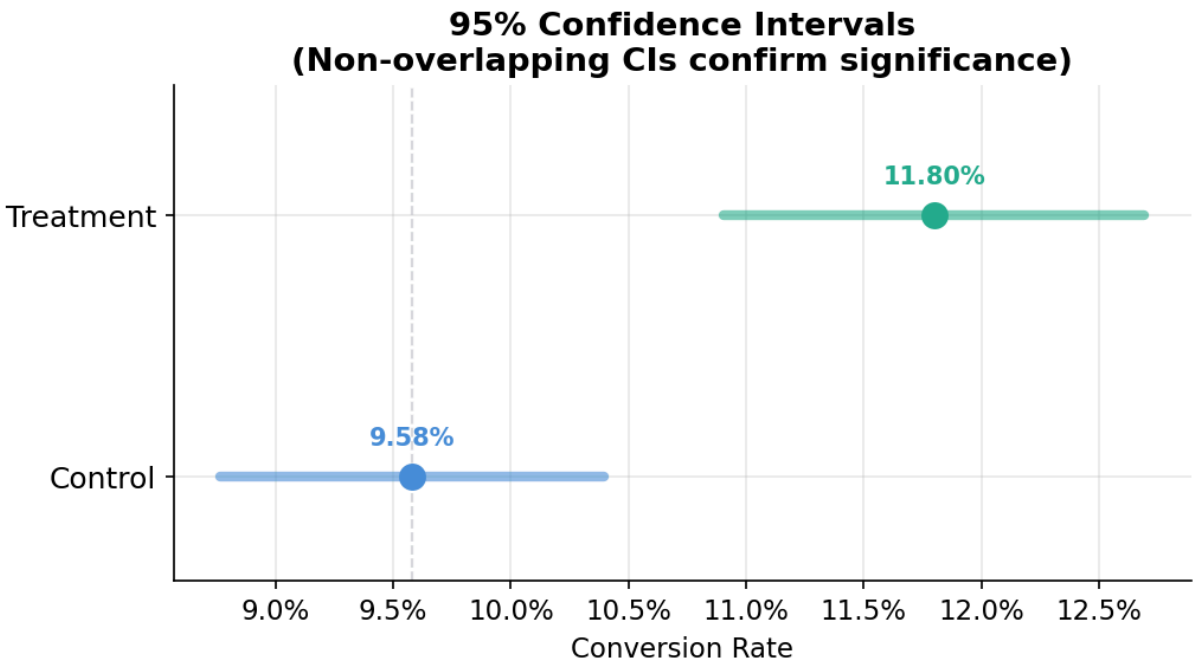



```
Saved: plot1_conversion_rates.png
```

In [35]:

```
fig, ax = plt.subplots(figsize=(7, 4))

y_positions = [1, 0]
labels      = ['Treatment', 'Control']
cis         = [ci_treatment, ci_control]
means       = [treatment['conversion_rate'], control['conversion_rate']]
colors      = [TEAL, BLUE]

for i, (ci, mean, color, label) in enumerate(zip(cis, means, colors, labels)):
    ax.plot(ci, [y_positions[i], y_positions[i]],
            color=color, linewidth=4, solid_capstyle='round', alpha=0.6)
    ax.plot(mean, y_positions[i], 'o', color=color, markersize=10, zorder=5)
    ax.text(mean, y_positions[i] + 0.12, f'{mean:.2%}',
            ha='center', fontsize=10, fontweight='bold', color=color)

ax.set_yticks(y_positions)
ax.set_yticklabels(labels, fontsize=12)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.1%}'))
ax.set_xlabel('Conversion Rate')
ax.set_title('95% Confidence Intervals\n(Non-overlapping CIs confirm significance)'
             fontweight='bold')
ax.set_ylim(-0.4, 1.5)
ax.axvline(x=control['conversion_rate'], color=GRAY, linestyle='--',
           linewidth=1, alpha=0.6)

plt.tight_layout()
plt.savefig('plot2_confidence_intervals.png', bbox_inches='tight')
```

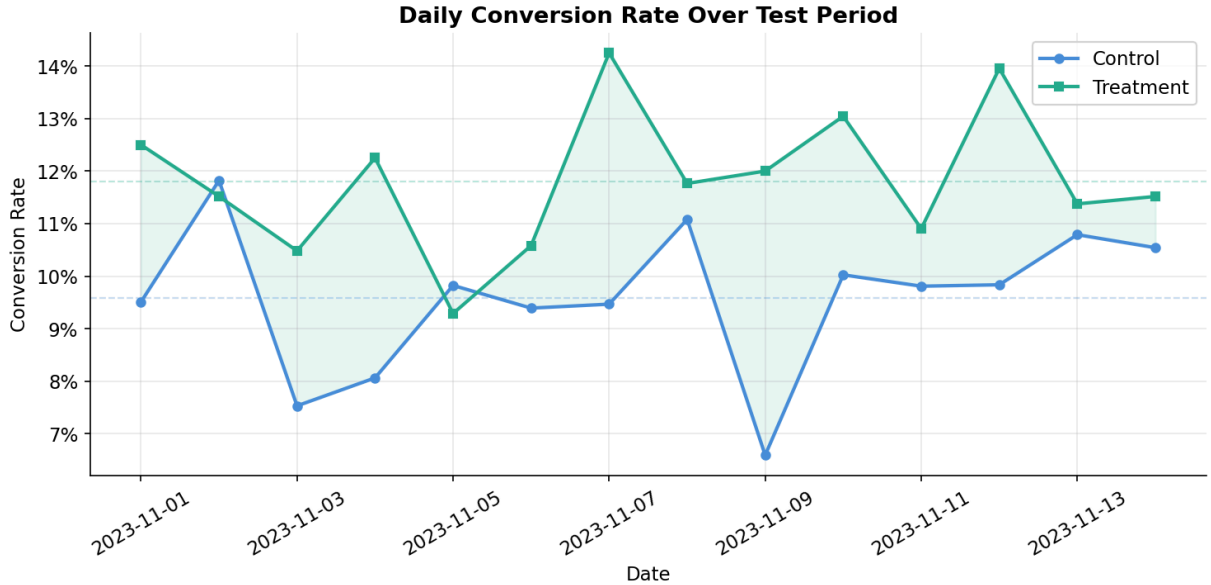
```
plt.show()
print("Saved: plot2_confidence_intervals.png")
```




```
Saved: plot2_confidence_intervals.png
```

In [36]:

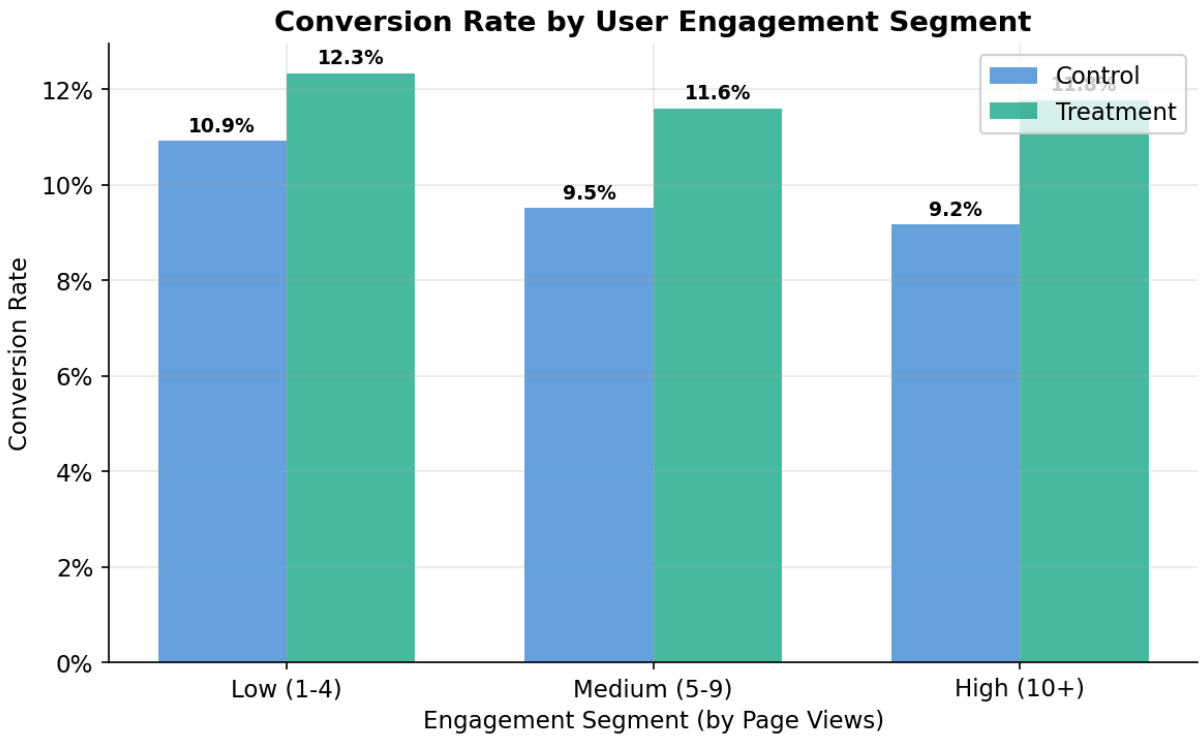
```
daily_pivot = daily.pivot(index='Date', columns='Group', values='conversion_rate').
daily_pivot = daily_pivot.sort_values('Date')

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(daily_pivot['Date'], daily_pivot['Control'],
        color=BLUE, marker='o', linewidth=2, label='Control', markersize=5)
ax.plot(daily_pivot['Date'], daily_pivot['Treatment'],
        color=TEAL, marker='s', linewidth=2, label='Treatment', markersize=5)

ax.fill_between(daily_pivot['Date'],
                daily_pivot['Control'], daily_pivot['Treatment'],
                alpha=0.1, color=TEAL)

ax.axhline(y=control['conversion_rate'],   color=BLUE, linestyle='--', alpha=0.3, l
ax.axhline(y=treatment['conversion_rate'], color=TEAL, linestyle='--', alpha=0.3, l

ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.set_xlabel('Date')
ax.set_ylabel('Conversion Rate')
ax.set_title('Daily Conversion Rate Over Test Period', fontweight='bold')
ax.legend()
plt.xticks(rotation=30)

plt.tight_layout()
plt.savefig('plot3_daily_trend.png', bbox_inches='tight')
plt.show()
print("Saved: plot3_daily_trend.png")
```




```
Saved: plot3_daily_trend.png
```

In [37]:

```
seg_pivot = engagement_seg.pivot(
    index='engagement', columns='Group', values='conversion_rate'
).reset_index()

x      = np.arange(len(seg_pivot))
width  = 0.35

fig, ax = plt.subplots(figsize=(8, 5))

b1 = ax.bar(x - width/2, seg_pivot['Control'],   width, color=BLUE, label='Control'
b2 = ax.bar(x + width/2, seg_pivot['Treatment'], width, color=TEAL, label='Treatmen

for bar in [*b1, *b2]:
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 0.002,
            f'{bar.get_height():.1%}',
            ha='center', fontsize=9, fontweight='bold')

ax.set_xticks(x)
ax.set_xticklabels(seg_pivot['engagement'])
ax.set_ylabel('Conversion Rate')
ax.set_xlabel('Engagement Segment (by Page Views)')
ax.set_title('Conversion Rate by User Engagement Segment', fontweight='bold')
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.legend()

plt.tight_layout()
plt.savefig('plot4_segments.png', bbox_inches='tight')
plt.show()
print("Saved: plot4_segments.png")
```

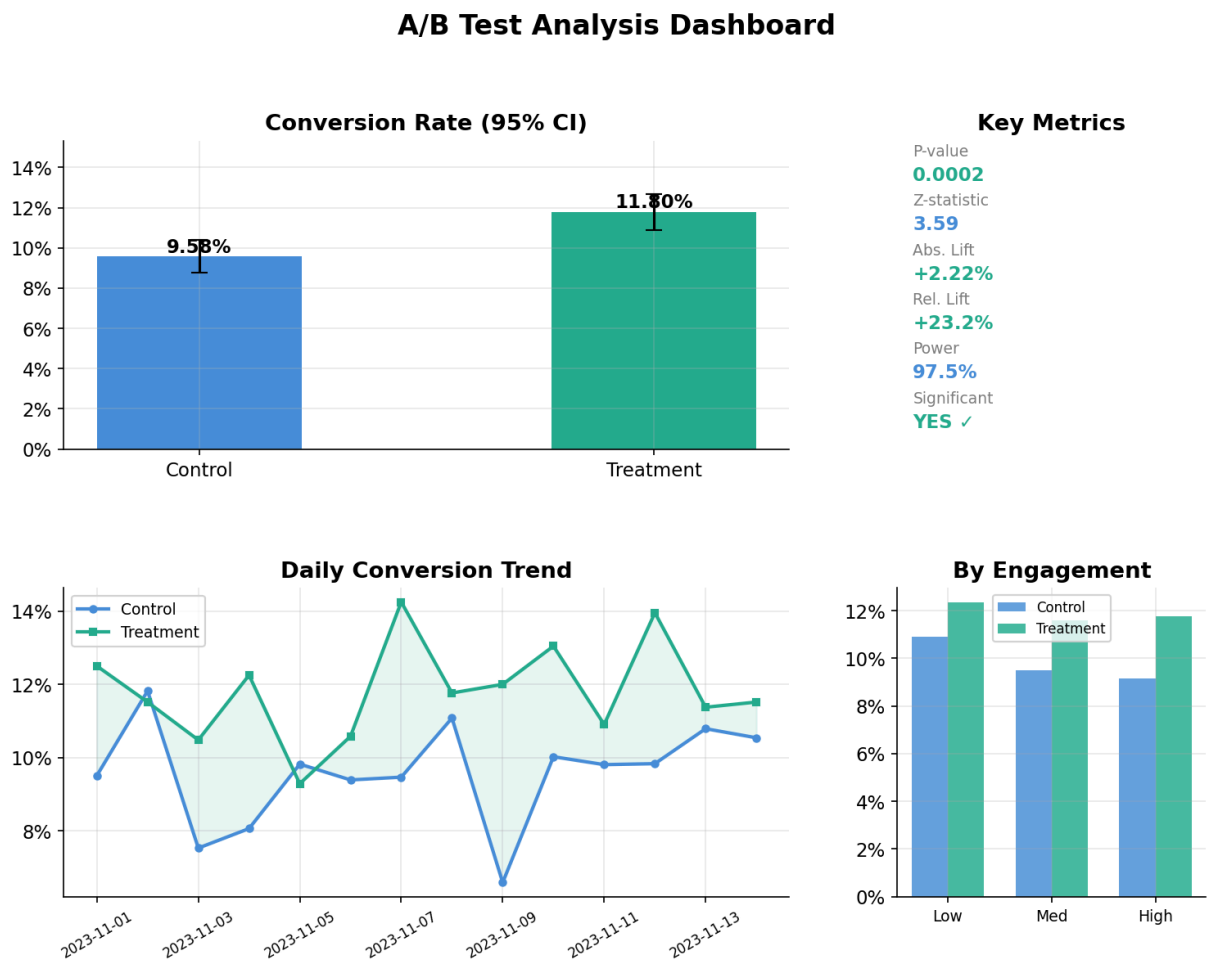



```
Saved: plot4_segments.png
```

In [38]:

```
fig = plt.figure(figsize=(12, 8))
fig.suptitle('A/B Test Analysis Dashboard', fontsize=16, fontweight='bold', y=1.01)
gs  = GridSpec(2, 3, figure=fig, hspace=0.45, wspace=0.35)

# Panel 1 — Bar chart
ax1 = fig.add_subplot(gs[0, :2])
bars = ax1.bar(groups, rates, color=[BLUE, TEAL], width=0.45,
               yerr=errors, capsize=5, error_kw=dict(elinewidth=1.5))
for bar, rate in zip(bars, rates):
    ax1.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.002,
             f'{rate:.2%}', ha='center', fontweight='bold')
ax1.set_title('Conversion Rate (95% CI)', fontweight='bold')
ax1.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax1.set_ylim(0, max(rates) * 1.3)

# Panel 2 — KPI scorecard
ax2 = fig.add_subplot(gs[0, 2])
ax2.axis('off')
kpis = [
    ('P-value',     f'{p_value:.4f}',  TEAL),
    ('Z-statistic', f'{z_stat:.2f}',   BLUE),
    ('Abs. Lift',   f'+{abs_diff:.2%}', TEAL),
    ('Rel. Lift',   f'+{rel_lift:.1f}%', TEAL),
    ('Power',       f'{power:.1%}',    BLUE),
    ('Significant', 'YES ✓',           TEAL),
]
for i, (label, value, color) in enumerate(kpis):
    ax2.text(0.05, 0.95 - i*0.16, label,  transform=ax2.transAxes,
             fontsize=9,  color='gray')
    ax2.text(0.05, 0.87 - i*0.16, value,  transform=ax2.transAxes,
             fontsize=11, color=color, fontweight='bold')
```

```
ax2.set_title('Key Metrics', fontweight='bold')

# Panel 3 — Daily trend
ax3 = fig.add_subplot(gs[1, :2])
ax3.plot(daily_pivot['Date'], daily_pivot['Control'],
         color=BLUE, marker='o', linewidth=2, label='Control', markersize=4)
ax3.plot(daily_pivot['Date'], daily_pivot['Treatment'],
         color=TEAL, marker='s', linewidth=2, label='Treatment', markersize=4)
ax3.fill_between(daily_pivot['Date'],
                 daily_pivot['Control'], daily_pivot['Treatment'],
                 alpha=0.1, color=TEAL)
ax3.set_title('Daily Conversion Trend', fontweight='bold')
ax3.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax3.legend(fontsize=9)
plt.setp(ax3.xaxis.get_majorticklabels(), rotation=30, fontsize=8)

# Panel 4 — Segments
ax4 = fig.add_subplot(gs[1, 2])
ax4.bar(x - width/2, seg_pivot['Control'],   width, color=BLUE, alpha=0.85, label='
ax4.bar(x + width/2, seg_pivot['Treatment'], width, color=TEAL, alpha=0.85, label='
ax4.set_xticks(x)
ax4.set_xticklabels(['Low', 'Med', 'High'], fontsize=9)
ax4.set_title('By Engagement', fontweight='bold')
ax4.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax4.legend(fontsize=8)

plt.savefig('plot5_dashboard.png', bbox_inches='tight', dpi=150)
plt.show()
print("Saved: plot5_dashboard.png")
```




```
Saved: plot5_dashboard.png
```

In [ ]: